

Sprint 3 Structured Notes: Strings and Text Processing

0. What Sprint 3 Is About

Sprint 3 is about working with **text** in JavaScript.

In JavaScript, text is called a **string**.

In Sprint 1, you learned how JavaScript can make a web page respond to actions.

In Sprint 2, you learned how JavaScript stores values with variables and how values have types.

Now, in Sprint 3, you learn how to create, combine, inspect, search, edit, and format text values.

The main goal of Sprint 3 is:

Use strings correctly and confidently in beginner JavaScript programs.

By the end of Sprint 3, you should understand:

- strings
- single quotes, double quotes, and backticks
- string immutability
- concatenation
- template literals
- string interpolation
- multiline strings
- `.length`
- bracket notation
- zero-based indexing
- escape characters
- newline characters
- `indexOf()`
- `includes()`
- `slice()`
- `trim()`
- `toUpperCase()`
- `toLowerCase()`
- `replace()`
- `repeat()`
- `charCodeAt()`
- `String.fromCharCode()`

- basic text-processing functions
 - common beginner mistakes with strings
-

Part A: String Basics

1. What Is a String?

A **string** is text.

Strings are used for values like:

- names
- emails
- usernames
- messages
- passwords
- button labels
- page titles
- form input
- sentences
- paragraphs

Examples:

```
const name = "Ada";  
const email = "ada@example.com";  
const message = "Welcome back!";  
const buttonText = "Submit";
```

Simple meaning:

A string is any text value written inside quotes.

2. Three Ways to Write Strings

JavaScript lets you write strings in three common ways.

2.1 Double Quotes

```
const first = "hello";
```

2.2 Single Quotes

```
const second = 'hello';
```

2.3 Backticks

```
const third = `hello`;
```

All three are strings.

```
console.log(typeof first); // string
console.log(typeof second); // string
console.log(typeof third); // string
```

Expected output:

```
string
string
string
```

Beginner rule:

Use one quote style consistently. Use backticks when you need variables inside text or multiline strings.

3. Number vs String

A number and a string can look similar, but they are not the same type.

```
const ageNumber = 25;
const ageString = "25";

console.log(typeof ageNumber);
console.log(typeof ageString);
```

Expected output:

```
number  
string
```

25 is a number.

"25" is text.

This matters because JavaScript behaves differently depending on the type.

```
console.log(25 + 5);    // 30  
console.log("25" + 5); // "255"
```

Why?

Because + can mean addition or string joining.

If one side is a string, JavaScript often joins the values as text.

Part B: String Immutability

4. What Does Immutable Mean?

Strings are **immutable**.

Simple meaning:

A string value cannot be changed in place.

String methods usually return a **new string** instead of changing the original string.

Example:

```
const name = "ada";  
  
name.toUpperCase();  
  
console.log(name);
```

Expected output:

```
ada
```

The original string did not change.

5. Correct Way to Use Returned Strings

If a method returns a new string, store the result in a variable.

```
const name = "ada";  
const upperName = name.toUpperCase();  
  
console.log(upperName);  
console.log(name);
```

Expected output:

```
ADA  
ada
```

The new string is stored in `upperName`.

The original string stays the same.

Beginner rule:

```
// This does not change the original string:  
name.toUpperCase();  
  
// This saves the new string:  
const fixedName = name.toUpperCase();
```

6. Why String Immutability Matters

String immutability matters because beginners often expect this:

```
const word = "hello";  
word.toUpperCase();  
console.log(word);
```

They may expect:

HELLO

But the real output is:

hello

Correct version:

```
const word = "hello";  
const updatedWord = word.toUpperCase();  
console.log(updatedWord);
```

Expected output:

HELLO

Part C: Combining Strings

7. Concatenation

Concatenation means joining strings together.

You can concatenate strings with `+`.

```
const firstName = "Ada";  
const lastName = "Lovelace";  
  
const fullName = firstName + " " + lastName;  
  
console.log(fullName);
```

Expected output:

```
Ada Lovelace
```

The space is also a string:

```
" "
```

Without it, the result would be:

```
AdaLovelace
```

8. Concatenating Strings with Variables

Example:

```
const name = "Ada";  
const course = "JavaScript";  
  
const message = "Hello, " + name + ". Welcome to " + course + ".";  
  
console.log(message);
```

Expected output:

```
Hello, Ada. Welcome to JavaScript.
```

This works, but it becomes hard to read when there are many variables.

9. The `+` Operator with Strings and Numbers

When `+` is used with a string, JavaScript often creates a string result.

```
console.log(5 + "10");
```

Expected output:

```
510
```

Why?

Because JavaScript treats the result as text.

`5` becomes `"5"`, then JavaScript joins:

```
"5" + "10" = "510"
```

Another example:

```
const score = 90;  
const message = "Score: " + score;  
  
console.log(message);
```

Expected output:

```
Score: 90
```

The number `90` becomes part of the text.

Part D: Template Literals

10. What Is a Template Literal?

A **template literal** is a string written with backticks.

```
const message = `Hello`;
```

Template literals are useful because they let you place variables directly inside a string.

11. String Interpolation

String interpolation means putting a variable or expression inside a string.

Use this pattern:

```
${value}
```

Example:

```
const name = "Ada";
const course = "JavaScript";

const message = `Hello, ${name}. Welcome to ${course}.`;

console.log(message);
```

Expected output:

```
Hello, Ada. Welcome to JavaScript.
```

Simple meaning:

`${name}` is replaced with the value stored in `name`.

12. Template Literals vs Concatenation

Concatenation version:

```
const name = "Ada";
const course = "JavaScript";

const message = "Hello, " + name + ". Welcome to " + course + ".";
```

Template literal version:

```
const name = "Ada";
const course = "JavaScript";

const message = `Hello, ${name}. Welcome to ${course}.`;
```

The template literal is usually easier to read.

Beginner rule:

Use template literals when you mix variables with text.

13. Expressions Inside Template Literals

You can put expressions inside `${}`.

```
const price = 10;
const quantity = 3;

const message = `Total: ${price * quantity}`;

console.log(message);
```

Expected output:

```
Total: 30
```

JavaScript calculates `price * quantity` first, then puts the result into the string.

14. Multiline Strings with Template Literals

Backticks also allow multiline strings.

```
const message = `Line 1
Line 2
Line 3`;

console.log(message);
```

Expected output:

```
Line 1
Line 2
Line 3
```

This is cleaner than manually using many newline characters.

Part E: String Length and Indexing

15. `.length`

Every string has a `.length` property.

`.length` tells you how many characters are in the string.

```
const word = "JavaScript";  
  
console.log(word.length);
```

Expected output:

```
10
```

The word `JavaScript` has 10 characters.

16. Bracket Notation

You can access one character from a string using bracket notation.

```
const word = "JavaScript";  
  
console.log(word[0]);  
console.log(word[1]);  
console.log(word[4]);
```

Expected output:

```
J  
a  
S
```

17. Zero-Based Indexing

Strings are **zero-indexed**.

Simple meaning:

The first character is at index `0`, not index `1`.

Example:

```
J a v a S c r i p t  
0 1 2 3 4 5 6 7 8 9
```

So:

```
const word = "JavaScript";  
  
console.log(word[0]); // J  
console.log(word[1]); // a  
console.log(word[2]); // v  
console.log(word[3]); // a  
console.log(word[4]); // S
```

18. Getting the Last Character

To get the last character, use `.length - 1`.

```
const word = "JavaScript";  
const lastIndex = word.length - 1;  
  
console.log(word[lastIndex]);
```

Expected output:

```
t
```

Why `length - 1`?

Because indexes start at `0`.

For `JavaScript`:

```
word.length // 10
```

Last index:

```
10 - 1 = 9
```

So:

```
word[9] // "t"
```

19. Accessing a Missing Index

If you ask for a character that does not exist, JavaScript returns `undefined`.

```
console.log("hello"[10]);
```

Expected output:

```
undefined
```

Why?

Because `"hello"` only has indexes `0` through `4`.

```
h e l l o  
0 1 2 3 4
```

There is no index `10`.

Part F: Escapes and Newlines

20. Newline Character

A newline means moving to the next line.

In a normal quoted string, use `\n`.

```
const message = "Line 1\nLine 2";  
  
console.log(message);
```

Expected output:

```
Line 1  
Line 2
```

Simple meaning:

`\n` means "new line."

21. Escaping Quotes

Sometimes you need quotes inside a string.

Problem example:

```
const sentence = "She said, "Hello".";
```

This breaks because JavaScript gets confused about where the string ends.

Correct version using backslash:

```
const sentence = "She said, \"Hello\".";   
  
console.log(sentence);
```

Expected output:

```
She said, "Hello".
```

The backslash tells JavaScript:

Treat this quote as part of the string, not the end of the string.

22. Avoiding Escapes with Different Quotes

You can also avoid escaping by using a different quote type.

```
const sentence = 'She said, "Hello".';  
  
console.log(sentence);
```

Expected output:

```
She said, "Hello".
```

Because the string starts and ends with single quotes, the double quotes inside are safe.

Another example:

```
const sentence = "It's a good day.";  
  
console.log(sentence);
```

Expected output:

```
It's a good day.
```

Because the string starts and ends with double quotes, the apostrophe inside is safe.

Part G: Searching Strings

23. `includes()`

Use `includes()` when you want a true-or-false answer.

```
const text = "JavaScript is fun";  
  
console.log(text.includes("fun"));  
console.log(text.includes("hard"));
```

Expected output:

```
true  
false
```

Simple meaning:

`includes()` checks whether a string contains another string.

24. `includes()` Is Case-Sensitive

`includes()` checks exact casing.

```
const text = "JavaScript";  
  
console.log(text.includes("Script"));  
console.log(text.includes("script"));
```

Expected output:

```
true  
false
```

`"Script"` and `"script"` are not the same.

To make a case-insensitive check, convert both strings to lowercase.

```
const text = "JavaScript";  
const search = "script";  
  
console.log(text.toLowerCase().includes(search.toLowerCase()));
```

Expected output:

```
true
```

25. `indexOf()`

Use `indexOf()` when you need the position of a substring.

```
const text = "freeCodeCamp";

console.log(text.indexOf("Code"));
console.log(text.indexOf("Camp"));
console.log(text.indexOf("Java"));
```

Expected output:

```
4
8
-1
```

Simple meaning:

`indexOf()` returns the index where the searched text begins.

If the text is not found, it returns `-1`.

26. `indexOf()` vs `includes()`

Use `includes()` when you only need yes or no.

```
const text = "JavaScript is fun";

console.log(text.includes("fun")); // true
```

Use `indexOf()` when you need the position.

```
const text = "freeCodeCamp";

console.log(text.indexOf("Code")); // 4
```

Comparison:

Method	Returns	Use when
<code>includes()</code>	<code>true</code> or <code>false</code>	You only need to know whether text exists
<code>indexOf()</code>	index number or <code>-1</code>	You need the position of the text

27. Checking `indexOf()` Correctly

Do not use `indexOf()` directly as a boolean condition.

Bad example:

```
if ("JavaScript".indexOf("Java")) {  
  console.log("Found");  
}
```

This is dangerous because:

```
"JavaScript".indexOf("Java")
```

returns:

```
0
```

In JavaScript, `0` behaves like false in conditions.

Correct version:

```
if ("JavaScript".indexOf("Java") !== -1) {  
  console.log("Found");  
}
```

Better beginner version:

```
if ("JavaScript".includes("Java")) {  
  console.log("Found");  
}
```

Part H: Extracting Text

28. `slice()`

`slice()` extracts part of a string.

```
const word = "JavaScript";  
  
console.log(word.slice(0, 4));  
console.log(word.slice(4));
```

Expected output:

```
Java  
Script
```

Simple meaning:

`slice()` takes a piece of a string and returns it as a new string.

29. How `slice(start, end)` Works

Syntax:

```
string.slice(startIndex, endIndex)
```

The `startIndex` is included.

The `endIndex` is not included.

Example:

```
const word = "JavaScript";  
  
console.log(word.slice(0, 4));
```

Indexes:

```
J a v a S c r i p t  
0 1 2 3 4 5 6 7 8 9
```

`slice(0, 4)` includes indexes `0`, `1`, `2`, and `3`.

It stops before index `4`.

Expected output:

Java

30. `slice()` with Only a Start Index

If you give only one number, `slice()` goes from that index to the end.

```
const word = "JavaScript";  
  
console.log(word.slice(4));
```

Expected output:

Script

It starts at index `4` and continues to the end.

31. Using `slice()` with Emails

Example:

```
const email = "ada@example.com";  
const atIndex = email.indexOf("@");  
const domain = email.slice(atIndex + 1);  
  
console.log(domain);
```

Expected output:

example.com

Step-by-step:

```
email.indexOf("@")
```

finds the position of `@`.

```
email.slice(atIndex + 1)
```

extracts everything after `@`.

Part I: Cleaning and Formatting Text

32. `trim()`

`trim()` removes whitespace from the beginning and end of a string.

Whitespace includes spaces, tabs, and newlines.

```
const rawName = "  Ada  ";
const cleanName = rawName.trim();

console.log(cleanName);
```

Expected output:

```
Ada
```

The original string is not changed.

```
console.log(rawName); // "  Ada  "
```

33. Why `trim()` Is Useful

Users often type extra spaces into forms.

Example:

```
const username = "  sangeeth  ";
const cleanUsername = username.trim();
```

```
console.log(cleanUsername);
```

Expected output:

```
sangeeth
```

Use `trim()` when cleaning user input.

34. `toUpperCase()`

`toUpperCase()` converts a string to uppercase.

```
const name = "ada";  
  
console.log(name.toUpperCase());
```

Expected output:

```
ADA
```

It returns a new string.

35. `toLowerCase()`

`toLowerCase()` converts a string to lowercase.

```
const email = "ADA@EXAMPLE.COM";  
  
console.log(email.toLowerCase());
```

Expected output:

```
ada@example.com
```

This is useful for normalizing emails, usernames, or search terms.

36. Case Normalization

Normalization means converting data into a consistent format.

Example:

```
const rawEmail = "  ADA@EXAMPLE.COM  ";
const cleanEmail = rawEmail.trim().toLowerCase();

console.log(cleanEmail);
```

Expected output:

```
ada@example.com
```

This combines:

```
trim()
```

and:

```
toLowerCase()
```

37. Method Chaining

Method chaining means calling one method after another.

```
const rawEmail = "  ADA@EXAMPLE.COM  ";
const cleanEmail = rawEmail.trim().toLowerCase();

console.log(cleanEmail);
```

Read it from left to right:

1. Start with `rawEmail`
2. Use `.trim()` to remove outside spaces
3. Use `.toLowerCase()` on the trimmed result
4. Store the final result in `cleanEmail`

Expected output:

```
ada@example.com
```

Part J: Replacing and Repeating Text

38. `replace()`

`replace()` replaces part of a string.

```
const text = "I like Java.";
const updated = text.replace("Java", "JavaScript");

console.log(updated);
```

Expected output:

```
I like JavaScript.
```

Simple meaning:

Replace `"Java"` with `"JavaScript"`.

39. `replace()` Returns a New String

Like other string methods, `replace()` does not change the original string.

```
const text = "I like Java.";
const updated = text.replace("Java", "JavaScript");

console.log(text);
console.log(updated);
```

Expected output:

```
I like Java.  
I like JavaScript.
```

40. `replace()` Replaces the First Match by Default

By default, `replace()` replaces the first matching part.

```
const text = "ha ha ha";  
const updated = text.replace("ha", "ho");  
  
console.log(updated);
```

Expected output:

```
ho ha ha
```

Only the first `ha` changed.

Later, when you learn regular expressions, you can replace more advanced patterns.

41. `repeat()`

`repeat()` repeats a string.

```
console.log("ha".repeat(3));
```

Expected output:

```
hahaha
```

Another example:

```
const line = "-".repeat(10);  
  
console.log(line);
```

Expected output:

```
-----
```

42. `repeat()` Rules

The repeat count must be:

- finite
- not negative
- a valid number

Valid:

```
console.log("x".repeat(3));
```

Invalid:

```
console.log("x".repeat(-1));
```

A negative repeat count causes an error.

Part K: ASCII and Character Codes

43. What Is ASCII?

ASCII is a system that gives characters numeric codes.

For example:

```
A -> 65  
B -> 66  
a -> 97  
b -> 98
```

Simple meaning:

Computers can represent characters with numbers.

JavaScript has methods that let you inspect and create characters using these codes.

44. `charCodeAt()`

`charCodeAt()` gives the character code at a specific index.

```
const letter = "A";  
  
console.log(letter.charCodeAt(0));
```

Expected output:

65

Why `0`?

Because the character `A` is at index `0`.

Another example:

```
const word = "JavaScript";  
  
console.log(word.charCodeAt(0));  
console.log(word.charCodeAt(4));
```

Expected output:

74
83

`J` has code `74`.

`S` has code `83`.

45. `String.fromCharCode()`

`String.fromCharCode()` creates a character from a character code.

```
console.log(String.fromCharCode(65));  
console.log(String.fromCharCode(66));  
console.log(String.fromCharCode(97));
```

Expected output:

```
A  
B  
a
```

Simple meaning:

Give JavaScript a number, and it returns the matching character.

46. `charCodeAt()` and `String.fromCharCode()` Together

```
const letter = "A";  
const code = letter.charCodeAt(0);  
const sameLetter = String.fromCharCode(code);  
  
console.log(code);  
console.log(sameLetter);
```

Expected output:

```
65  
A
```

Step-by-step:

1. Start with `"A"`
2. Get its character code with `charCodeAt(0)`
3. Convert the code back to a character with `String.fromCharCode(code)`

Part L: Core Sprint 3 Code Pattern

47. Core Code Pattern

Type this code yourself.

Before running it, predict the output.

```
const name = "Ada";
const greeting = `Hello, ${name}!`;

const message = " JavaScript is fun ";

console.log(greeting);
console.log(message.trim().toUpperCase());
console.log("freeCodeCamp".indexOf("Code"));
console.log("ha".repeat(3));
```

Expected output:

```
Hello, Ada!
JAVASCRIPT IS FUN
4
hahaha
```

48. Core Code Pattern Explanation

This line stores a string:

```
const name = "Ada";
```

This line creates a template literal:

```
const greeting = `Hello, ${name}!`;
```

`${name}` becomes `Ada`.

So `greeting` becomes:

```
Hello, Ada!
```

This line stores text with extra spaces:

```
const message = " JavaScript is fun ";
```

This line trims the spaces and converts the result to uppercase:

```
console.log(message.trim().toUpperCase());
```

`message.trim()` creates:

```
JavaScript is fun
```

Then `.toUpperCase()` creates:

```
JAVASCRIPT IS FUN
```

This line finds the position of `"Code"`:

```
console.log("freeCodeCamp".indexOf("Code"));
```

Expected output:

```
4
```

This line repeats `"ha"` three times:

```
console.log("ha".repeat(3));
```

Expected output:

```
hahaha
```

Part M: Practical String Functions

49. Function: `cleanUsername(name)`

Goal:

Take messy user input and clean it.

```
function cleanUsername(name) {  
  return name.trim().toLowerCase();  
}  
  
console.log(cleanUsername(" Sangeeth "));  
console.log(cleanUsername(" ADA_LOVELACE "));
```

Expected output:

```
sangeeth  
ada_lovelace
```

This function uses:

- `trim()`
- `toLowerCase()`
- `return`

Simple meaning:

Remove extra outside spaces and make the username lowercase.

50. Function: `formatFullName(first, last)`

Goal:

Clean two names and combine them.

```
function formatFullName(first, last) {  
  const cleanFirst = first.trim();  
  const cleanLast = last.trim();  
  
  return `${cleanFirst} ${cleanLast}`;  
}
```

```
}  
  
console.log(formatFullName(" Ada ", " Lovelace "));
```

Expected output:

```
Ada Lovelace
```

This function uses:

- parameters
- variables
- `trim()`
- template literals
- `return`

51. Function: `containsWord(text, word)`

Goal:

Check whether one string contains another string.

```
function containsWord(text, word) {  
  return text.toLowerCase().includes(word.toLowerCase());  
}  
  
console.log(containsWord("JavaScript is fun", "script"));  
console.log(containsWord("JavaScript is fun", "Python"));
```

Expected output:

```
true  
false
```

Why use `toLowerCase()`?

Because string searches are case-sensitive.

This would be false:

```
"JavaScript".includes("script")
```

But this is true:

```
"JavaScript".toLowerCase().includes("script".toLowerCase())
```

52. Function: `extractDomain(email)`

Goal:

Extract the domain from an email address.

```
function extractDomain(email) {  
  const atIndex = email.indexOf("@");  
  return email.slice(atIndex + 1);  
}  
  
console.log(extractDomain("ada@example.com"));
```

Expected output:

```
example.com
```

Step-by-step:

1. Find the `@` symbol
2. Start slicing after `@`
3. Return the domain

53. Function: `maskEmail(email)`

Goal:

Turn this:

```
ada@example.com
```

into this:

```
a**@example.com
```

Code:

```
function maskEmail(email) {  
  const atIndex = email.indexOf("@");  
  const firstLetter = email[0];  
  const domain = email.slice(atIndex);  
  
  return `${firstLetter}**${domain}`;  
}  
  
console.log(maskEmail("ada@example.com"));
```

Expected output:

```
a**@example.com
```

Step-by-step:

```
const atIndex = email.indexOf("@");
```

Finds where `@` is.

```
const firstLetter = email[0];
```

Gets the first character of the email.

```
const domain = email.slice(atIndex);
```

Gets everything from `@` onward.

```
return `${firstLetter}**${domain}`;
```

Builds the masked email.

Part N: Sprint 3 Practice

54. Warm-Up Drill 1: Create Strings

Create strings with double quotes, single quotes, and backticks.

```
const doubleQuoteString = "hello";
const singleQuoteString = 'hello';
const backtickString = `hello`;

console.log(doubleQuoteString);
console.log(singleQuoteString);
console.log(backtickString);
```

Expected output:

```
hello
hello
hello
```

55. Warm-Up Drill 2: Practice String Methods

Practice these methods:

- `.length`
- bracket notation
- `indexOf()`
- `includes()`
- `slice()`
- `trim()`
- `toUpperCase()`
- `toLowerCase()`
- `replace()`
- `repeat()`

Code:

```
const text = " JavaScript is fun ";

console.log(text.length);
console.log(text[2]);
```

```
console.log(text.indexOf("Script"));
console.log(text.includes("fun"));
console.log(text.slice(2, 12));
console.log(text.trim());
console.log(text.toUpperCase());
console.log(text.toLowerCase());
console.log(text.replace("fun", "powerful"));
console.log("ha".repeat(3));
```

Before running the code, predict each output.

56. Warm-Up Drill 3: Clean Names

Task:

Convert names with extra spaces into clean display strings.

```
const firstName = "  ada  ";
const secondName = "  grace  ";
const thirdName = "  linus  ";

console.log(firstName.trim());
console.log(secondName.trim());
console.log(thirdName.trim());
```

Expected output:

```
ada
grace
linus
```

Extra improvement:

```
function cleanName(name) {
  const trimmed = name.trim();
  const firstLetter = trimmed[0].toUpperCase();
  const rest = trimmed.slice(1).toLowerCase();

  return `${firstLetter}${rest}`;
}

console.log(cleanName("  ada  "));
```

```
console.log(cleanName(" GRACE "));  
console.log(cleanName("  linus  "));
```

Expected output:

```
Ada  
Grace  
Linus
```

Part O: Starter Code

57. Starter Code

Type this code yourself.

```
const rawName = " ada lovelace ";  
const cleaned = rawName.trim();  
const message = `Hello, ${cleaned.toUpperCase()}!`;  
  
console.log(cleaned[0]);  
console.log(cleaned.includes("love"));  
console.log(message);
```

Expected output:

```
a  
true  
Hello, ADA LOVELACE!
```

58. Starter Code Explanation

This line stores a string with extra spaces:

```
const rawName = " ada lovelace ";
```

This line removes the outside spaces:

```
const cleaned = rawName.trim();
```

Now `cleaned` is:

```
ada lovelace
```

This line creates a message:

```
const message = `Hello, ${cleaned.toUpperCase()}!`;
```

`cleaned.toUpperCase()` becomes:

```
ADA LOVELACE
```

So `message` becomes:

```
Hello, ADA LOVELACE!
```

This line gets the first character:

```
console.log(cleaned[0]);
```

Expected output:

```
a
```

This line checks whether `cleaned` contains `"love"`:

```
console.log(cleaned.includes("love"));
```

Expected output:

```
true
```

This line prints the final message:

```
console.log(message);
```

Expected output:

```
Hello, ADA LOVELACE!
```

Part P: Debug Challenge

59. Debug Challenge: Missing String Index

Code:

```
console.log("hello"[10]);
```

Task:

Predict why the result is `undefined`.

Explanation:

The string `"hello"` has five characters.

Indexes:

```
h e l l o  
0 1 2 3 4
```

There is no index `10`.

So JavaScript returns:

```
undefined
```

This is not a crash.

It means:

There is no character at that position.

Part Q: Mini-Builds

60. Mini-Build 1: `cleanUsername(name)`

Task:

Create a function that trims whitespace and converts the result to lowercase.

Solution:

```
function cleanUsername(name) {  
  return name.trim().toLowerCase();  
}  
  
console.log(cleanUsername(" Sangeeth "));  
console.log(cleanUsername(" ADA "));  
console.log(cleanUsername(" JavaScript_User "));
```

Expected output:

```
sangeeth  
ada  
javascript_user
```

61. Mini-Build 2: `maskEmail(email)`

Task:

Create a function that turns:

```
ada@example.com
```

into:

```
a**@example.com
```

Solution:

```
function maskEmail(email) {  
  const atIndex = email.indexOf("@");  
  const firstLetter = email[0];  
  const domain = email.slice(atIndex);  
  
  return `${firstLetter}**${domain}`;  
}  
  
console.log(maskEmail("ada@example.com"));  
console.log(maskEmail("sangeeth@example.com"));
```

Expected output:

```
a**@example.com  
s**@example.com
```

62. Mini-Build 3: `containsText(text, searchTerm)`

Task:

Create a function that checks whether text contains a search term.

Make the check case-insensitive.

```
function containsText(text, searchTerm) {  
  return text.toLowerCase().includes(searchTerm.toLowerCase());  
}  
  
console.log(containsText("JavaScript is fun", "script"));  
console.log(containsText("JavaScript is fun", "PYTHON"));
```

Expected output:

```
true  
false
```

63. Mini-Build 4: `getInitials(firstName, lastName)`

Task:

Create initials from two names.

```
function getInitials(firstName, lastName) {  
  const firstInitial = firstName.trim()[0].toUpperCase();  
  const lastInitial = lastName.trim()[0].toUpperCase();  
  
  return `${firstInitial}.${lastInitial}.`;  
}  
  
console.log(getInitials(" ada ", " lovelace "));
```

Expected output:

A.L.

Part R: Common Sprint 3 Mistakes

64. Mistake 1: Thinking String Methods Change the Original String

Wrong expectation:

```
const name = "ada";  
name.toUpperCase();  
  
console.log(name);
```

Actual output:

ada

Correct:

```
const name = "ada";  
const upperName = name.toUpperCase();  
  
console.log(upperName);
```

Expected output:

ADA

65. Mistake 2: Forgetting Indexes Start at 0

Wrong idea:

The first character is at index `1`.

Correct idea:

The first character is at index `0`.

Example:

```
const word = "hello";  
  
console.log(word[0]);  
console.log(word[1]);
```

Expected output:

```
h  
e
```

66. Mistake 3: Using `indexOf()` Like a Boolean

Bad:

```
if ("JavaScript".indexOf("Java")) {  
  console.log("Found");  
}
```

Problem:

`"JavaScript".indexOf("Java")` returns `0`.

`0` behaves like false in a condition.

Correct:

```
if ("JavaScript".indexOf("Java") !== -1) {  
  console.log("Found");  
}
```

Simpler:

```
if ("JavaScript".includes("Java")) {  
  console.log("Found");  
}
```

67. Mistake 4: Forgetting That Searches Are Case-Sensitive

```
console.log("JavaScript".includes("script"));
```

Expected output:

```
false
```

Why?

Because `"Script"` and `"script"` are different.

Correct case-insensitive version:

```
const text = "JavaScript";  
const search = "script";  
  
console.log(text.toLowerCase().includes(search.toLowerCase()));
```

Expected output:

```
true
```

68. Mistake 5: Forgetting That `slice()` Does Not Include the End Index

```
const word = "JavaScript";  
  
console.log(word.slice(0, 4));
```

Expected output:

```
Java
```

It does not include index `4`.

It stops before index `4`.

69. Mistake 6: Expecting `replace()` to Replace Every Match

```
const text = "ha ha ha";  
const updated = text.replace("ha", "ho");  
  
console.log(updated);
```

Expected output:

```
ho ha ha
```

Only the first match is replaced by default.

Part S: Mental Models

70. Mental Model: String as a Row of Characters

Think of a string like a row of characters.

```
J a v a S c r i p t  
0 1 2 3 4 5 6 7 8 9
```

Each character has a position.

The first position is `0`.

71. Mental Model: String Methods Return New Strings

Think of string methods like machines.

Input:

```
" ada "
```

Method:

```
trim()
```

Output:

```
"ada"
```

The machine gives you a new value.

It does not rewrite the original value.

72. Mental Model: Template Literals Fill in Blanks

Template literals are like sentences with blanks.

```
const name = "Ada";  
const message = `Hello, ${name}!`;
```

JavaScript fills the blank:

```
Hello, Ada!
```

73. Mental Model: `includes()` vs `indexOf()`

`includes()` asks:

Is this text inside the string?

`indexOf()` asks:

Where does this text start?

Example:

```
const text = "freeCodeCamp";  
  
console.log(text.includes("Code"));  
console.log(text.indexOf("Code"));
```

Expected output:

```
true  
4
```

Part T: Key Vocabulary

74. Vocabulary

String

Text data in JavaScript.

```
"hello"
```

Immutable

Cannot be changed in place.

Strings are immutable.

Concatenation

Joining strings together.

```
"Hello " + "Ada"
```

Template Literal

A string written with backticks.

```
`Hello`
```

Interpolation

Putting variables or expressions inside a template literal.

```
`Hello, ${name}`
```

`.length`

A property that tells how many characters are in a string.

```
"hello".length // 5
```

Bracket Notation

A way to access one character by index.

```
"hello"[0] // "h"
```

Index

A position number.

The first index is `0`.

Escape Character

A backslash used to give special meaning to a character.

```
"She said, \"Hello\"."
```

Newline

A line break.

```
"Line 1\nLine 2"
```

`includes()`

Checks whether text exists inside a string.

Returns `true` or `false`.

`indexOf()`

Finds the starting position of text.

Returns an index or `-1`.

`slice()`

Extracts part of a string.

`trim()`

Removes whitespace from the beginning and end.

`toUpperCase()`

Converts text to uppercase.

`toLowerCase()`

Converts text to lowercase.

`replace()`

Replaces part of a string.

`repeat()`

Repeats a string a chosen number of times.

ASCII

A system that represents characters as numbers.

`charCodeAt()`

Gets the numeric code of a character.

`String.fromCharCode()`

Creates a character from a numeric code.

Part U: Revision Questions

75. Sprint 3 Revision Questions

Try to answer these without looking at the notes.

1. What is a string?
2. What are three ways to write strings in JavaScript?
3. What does it mean that strings are immutable?
4. Why does `name.toUpperCase()` not change the original string?
5. What is concatenation?
6. What happens when you use `+` with a string and a number?
7. What is a template literal?
8. What does `${name}` do inside a template literal?
9. What does `.length` return?
10. What does zero-indexed mean?
11. How do you get the first character of a string?
12. How do you get the last character of a string?
13. What does `"hello"[10]` return?
14. How do you create a newline inside a string?
15. How do you put quotes inside a string?
16. What does `includes()` return?
17. What does `indexOf()` return when text is found?
18. What does `indexOf()` return when text is not found?
19. When should you use `includes()` instead of `indexOf()`?
20. What does `slice(0, 4)` do?
21. Does `slice()` include the ending index?
22. What does `trim()` do?
23. What does `toUpperCase()` do?
24. What does `toLowerCase()` do?
25. What does `replace()` do?
26. Does `replace()` change the original string?

27. What does `repeat()` do?
 28. What does `charCodeAt()` do?
 29. What does `String.fromCharCode()` do?
 30. Why are string searches case-sensitive?
-

Part V: Self-Check Answers

76. Answers

1. What is a string?

A string is text data in JavaScript.

Example:

```
"hello"
```

2. What are three ways to write strings?

```
"hello"  
'hello'  
`hello`
```

3. What does immutable mean?

Immutable means the original value cannot be changed in place.

String methods return new strings.

4. Why does `name.toUpperCase()` not change the original string?

Because strings are immutable.

You must store the returned value.

```
const upperName = name.toUpperCase();
```

5. What is concatenation?

Concatenation means joining strings together.

```
"Hello " + "Ada"
```

6. What happens when you use `+` with a string and a number?

JavaScript usually converts the number to text and joins the values.

```
5 + "10" // "510"
```

7. What is a template literal?

A template literal is a string written with backticks.

```
`Hello`
```

8. What does `${name}` do?

It inserts the value of `name` into the string.

9. What does `.length` return?

It returns the number of characters in the string.

10. What does zero-indexed mean?

The first character is at index `0`.

11. How do you get the first character?

```
text[0]
```

12. How do you get the last character?

```
text[text.length - 1]
```

13. What does `"hello"[10]` return?

It returns `undefined` because there is no character at index `10`.

14. How do you create a newline?

Use `\n`.

```
"Line 1\nLine 2"
```

15. How do you put quotes inside a string?

Use escaping:

```
"She said, \"Hello\"."
```

Or use a different quote type:

```
'She said, "Hello".'
```

16. What does `includes()` return?

It returns `true` or `false`.

17. What does `indexOf()` return when text is found?

It returns the starting index.

18. What does `indexOf()` return when text is not found?

It returns `-1`.

19. When should you use `includes()`?

Use it when you only need yes or no.

20. What does `slice(0, 4)` do?

It extracts characters from index `0` up to, but not including, index `4`.

21. Does `slice()` include the ending index?

No.

22. What does `trim()` do?

It removes whitespace from the beginning and end of a string.

23. What does `toUpperCase()` do?

It returns an uppercase version of the string.

24. What does `toLowerCase()` do?

It returns a lowercase version of the string.

25. What does `replace()` do?

It replaces part of a string and returns a new string.

26. Does `replace()` change the original string?

No.

27. What does `repeat()` do?

It repeats a string.

28. What does `charCodeAt()` do?

It returns the character code at an index.

29. What does `String.fromCharCode()` do?

It creates a character from a character code.

30. Why are string searches case-sensitive?

Because uppercase and lowercase letters are different characters.

Part W: Final Sprint 3 Checklist

77. Final Checklist

You are ready to move on from Sprint 3 when you can do these things:

- ☐ Explain what a string is.
- ☐ Create strings with double quotes, single quotes, and backticks.
- ☐ Explain string immutability.
- ☐ Explain why string methods return new strings.
- ☐ Combine strings with `+`.
- ☐ Use template literals with `${}`.
- ☐ Create multiline strings.
- ☐ Use `.length`.
- ☐ Access characters with bracket notation.
- ☐ Explain zero-based indexing.

- [] Handle missing indexes returning `undefined`.
- [] Use `\n` for newlines.
- [] Escape quotes inside strings.
- [] Use `includes()` for yes/no text checks.
- [] Use `indexOf()` to find text position.
- [] Correctly check `indexOf()` with `!== -1`.
- [] Use `slice()` to extract part of a string.
- [] Use `trim()` to clean outside spaces.
- [] Use `toUpperCase()` and `toLowerCase()`.
- [] Use `replace()` to replace text.
- [] Use `repeat()` to repeat text.
- [] Use `charCodeAt()` to get character codes.
- [] Use `String.fromCharCode()` to create characters from codes.
- [] Build `cleanUsername(name)`.
- [] Build `formatFullName(first, last)`.
- [] Build `containsWord(text, word)`.
- [] Build `extractDomain(email)`.
- [] Build `maskEmail(email)`.
- [] Explain common string mistakes.

Part X: Minimum Passing Standard

78. Minimum Passing Standard for Sprint 3

You understand Sprint 3 if you can clearly explain and use this code:

```
const rawName = " ada lovelace ";
const cleaned = rawName.trim();
const message = `Hello, ${cleaned.toUpperCase()}!`;

console.log(cleaned[0]);
console.log(cleaned.includes("love"));
console.log(message);
```

Expected output:

```
a
true
Hello, ADA LOVELACE!
```

You should be able to explain:

1. `rawName` stores a string.
 2. `trim()` removes outside spaces.
 3. `cleaned[0]` gets the first character.
 4. `includes("love")` checks whether the text exists.
 5. `toUpperCase()` returns a new uppercase string.
 6. The template literal inserts the uppercase name into the message.
-

Part Y: One-Sentence Summary

79. Sprint 3 One-Sentence Summary

Sprint 3 teaches you how to work with text in JavaScript by creating strings, combining them, checking their characters, searching inside them, extracting parts, cleaning user input, formatting output, replacing text, repeating text, and using basic character codes.